Table of Contents

1 A Practical Guide to Vector Conventions in Deep Learning **2**

Bibliography **9**

Chapter 1

A Practical Guide to Vector Conventions in Deep Learning

One subtle source of confusion when working with Deep Learning is the way vectors are represented. In most mathematics and linear algebra textbooks, vectors are represented as **column vectors**:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

However, in Deep Learning libraries such as TensorFlow and PyTorch, and in many publications, vectors are often represented as **row vectors**:

$$\mathbf{x} = [x_1, x_2, \ldots, x_n]$$

Although these two representations contain exactly the same information, they lead to different matrix expressions and different interpretations of matrix dimensions.
Understanding how to move between the two conventions is important when reading the literature, implementing algorithms, or translating mathematical derivations into code.

- **Column-vector convention**
  Vectors are represented as columns. The product of a matrix $\mathbf{A}$ and a vector $\mathbf{x}$ is computed by placing the matrix on the left and the vector on the right:

$$\mathbf{y} = \mathbf{A}\mathbf{x} \tag{1}$$

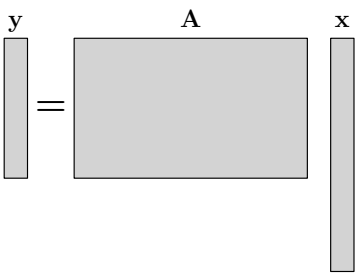

Figure 1.1: Matrix-vector multiplication using the column-vector convention.

- **Row-vector convention**
  Vectors are represented as rows (in the following, the symbol $\sim$ is used to denote quantities written in the row-vector convention). The product of a matrix $\widetilde{\mathbf{A}}$ and a vector $\widetilde{\mathbf{x}}$ is computed by placing the vector on the left and the matrix on the right:

$$\widetilde{\mathbf{y}} = \widetilde{\mathbf{x}}\widetilde{\mathbf{A}} \tag{2}$$

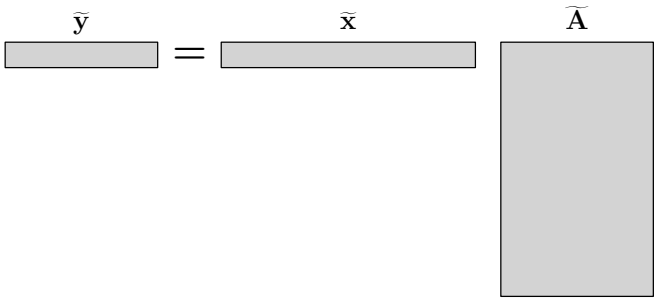

Figure 1.2: Matrix-vector multiplication using the row-vector convention.

Important: No matter which convention is adopted, the underlying multiplication rules are the same: an "horizontal" vector is always placed on the left and a "vertical" vector on the right. What varies from one convention to another is only in how these vectors are named and interpreted.

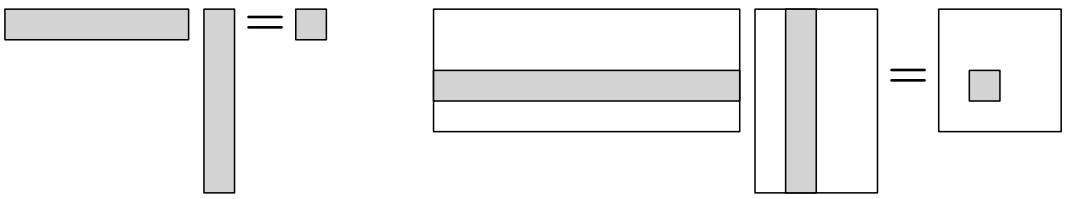

Figure 1.3: xxxxx

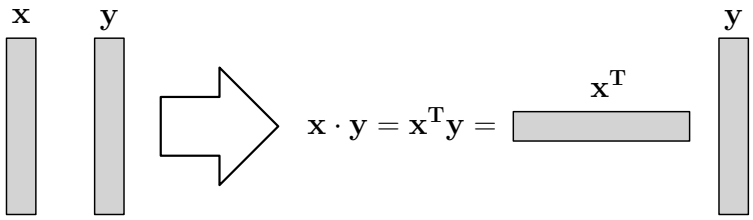

Figure 1.4: xxxxx.

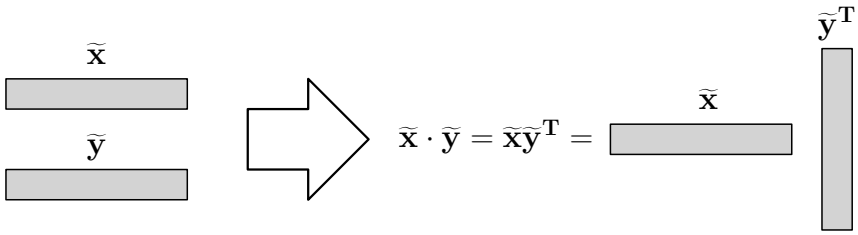

Figure 1.5: xxxxx.

The two expressions (1) and (2) describe exactly the same operation. They are related through transposition:

$$\widetilde{\mathbf{x}} = \mathbf{x}^T, \qquad \widetilde{\mathbf{A}} = \mathbf{A}^T, \qquad \widetilde{\mathbf{y}} = \mathbf{y}^T.$$

Therefore,

$$\mathbf{y} = \mathbf{A}\mathbf{x}$$

implies

$$\mathbf{y}^T = (\mathbf{A}\mathbf{x})^T = \mathbf{x}^T\mathbf{A}^T$$

or equivalently

$$\widetilde{\mathbf{y}} = \widetilde{\mathbf{x}}\widetilde{\mathbf{A}}$$

Thus, switching convention amounts to transposing every vector and matrix and reversing the order of multiplication.
The following transposition rule applies to any product of matrices:

$$(A_1 A_2 \cdots A_n)^T = A_n^T \cdots A_2^T A_1^T \tag{3}$$

Matrix Assembly

The choice of vector convention affects how collections of vectors are assembled into matrices.

- **Column-vector convention**
  Suppose we have N vectors

  $$\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_N$$

  The vectors are assembled as columns:

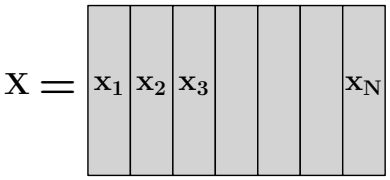

Figure 1.6: xxxxx.

- **Row-vector convention**
  Suppose we have N vectors

  $$\widetilde{\mathbf{x}}_1, \widetilde{\mathbf{x}}_2, \ldots, \widetilde{\mathbf{x}}_N$$

  They are assembled as rows:

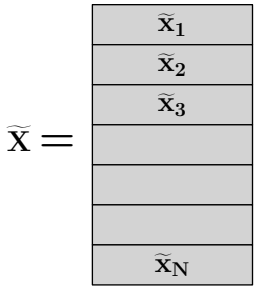

Figure 1.7: xxxxx.

Example: Feed-Forward Neural Network

Consider a fully connected layer (also called a dense layer).
Using the column-vector convention,

$$\mathbf{y} = \mathbf{A}\mathbf{x} + \mathbf{b}$$

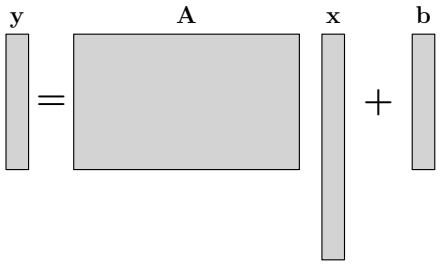

Figure 1.8: xxxxx.

Taking the transpose,

$$\mathbf{y}^T = (\mathbf{A}\mathbf{x} + \mathbf{b})^T$$

which gives

$$\mathbf{y}^T = \mathbf{x}^T\mathbf{A}^T + \mathbf{b}^T$$

Therefore,

$$\widetilde{\mathbf{y}} = \widetilde{\mathbf{x}}\widetilde{\mathbf{A}} + \widetilde{\mathbf{b}}$$

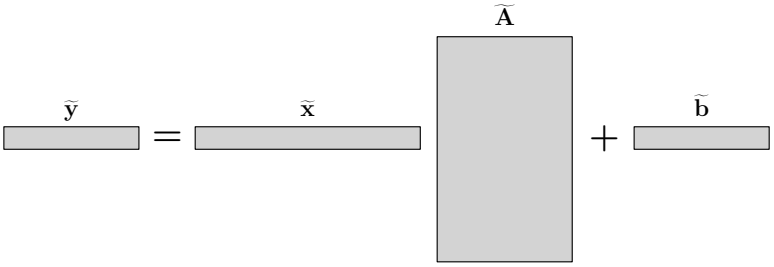

Figure 1.9: xxxxx.

The two equations represent exactly the same neural-network layer written using different conventions.

Example: Scaled Dot-Product Attention

The scaled dot-product attention mechanism was introduced in Vaswani et al.'s paper *Attention Is All You Need*.

- **Row-vector convention**
  This is the convention used in the paper.

$$\text{Attention}(\widetilde{\mathbf{Q}}, \widetilde{\mathbf{K}}, \widetilde{\mathbf{V}}) = \text{Softmax}\left(\frac{\widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^T}{\sqrt{d_k}}\right)\widetilde{\mathbf{V}} \tag{4}$$

  Assuming

$$\widetilde{\mathbf{Q}} \in \mathbb{R}^{d_c \times d_k} \qquad \widetilde{\mathbf{K}} \in \mathbb{R}^{d_c \times d_k} \qquad \widetilde{\mathbf{V}} \in \mathbb{R}^{d_c \times d_v}$$

  the score matrix is

$$\widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^T \in \mathbb{R}^{d_c \times d_c}$$

  After applying the softmax operation (row by row), the attention weights remain of size

$$d_c \times d_c$$

  The final attention output has dimensions

$$d_c \times d_v$$

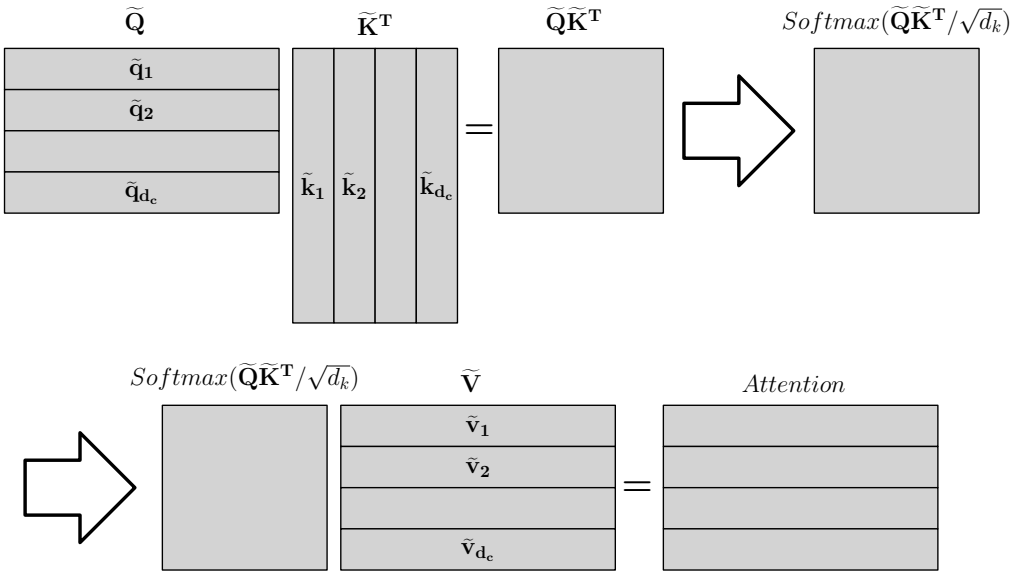

Figure 1.10: xxxxx.

- **Column-vector convention**
  By taking the transpose of equation (4) and using (3) we obtain:

$$\text{Attention}(\mathbf{Q},\mathbf{K},\mathbf{V}) = \mathbf{V}\ \text{Softmax}\left(\frac{\mathbf{K}^T\mathbf{Q}}{\sqrt{d_k}}\right)$$

  where the Softmax operation is applied column by column. Here,

$$\mathbf{Q} \in \mathbb{R}^{d_k \times d_c} \qquad \mathbf{K} \in \mathbb{R}^{d_k \times d_c} \qquad \mathbf{V} \in \mathbb{R}^{d_v \times d_c}$$

  The matrix

$$\mathbf{K}^T\mathbf{Q}$$

  has dimensions

$$d_c \times d_c$$

  and the attention output has dimensions

$$d_v \times d_c$$

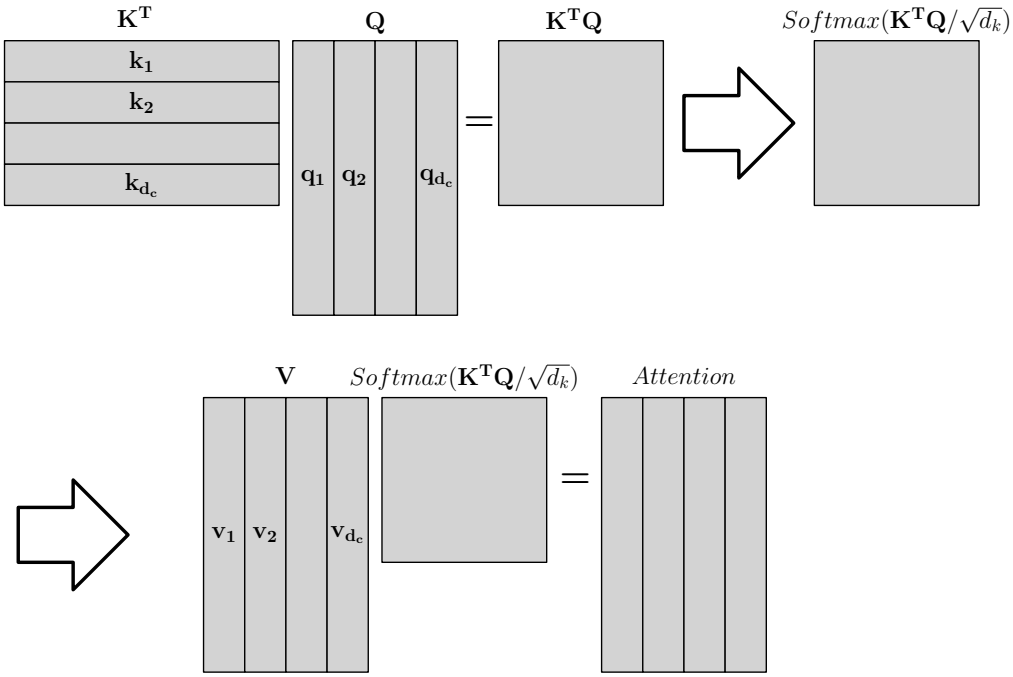

Figure 1.11: xxxxx.

Bibliography